



南京大學
NANJING UNIVERSITY

六、通用同步/异步收发器 (USART)

主讲教师：闫浩
机器人与自动化学院



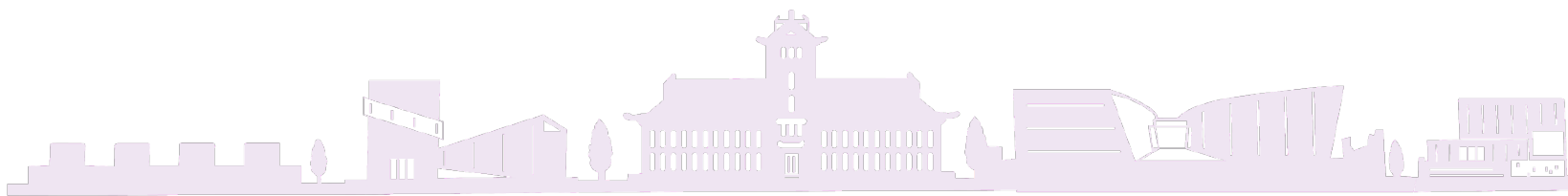
目录

CONTENTS

01 通信基本概念

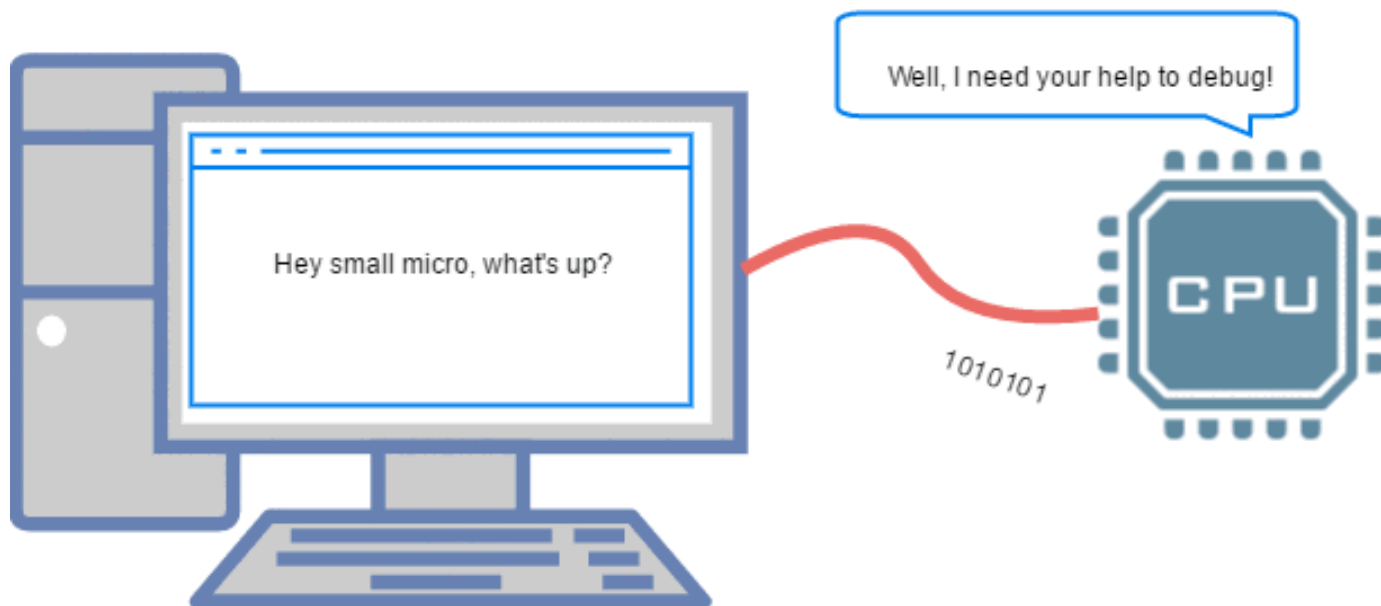
02 STM32串口系统

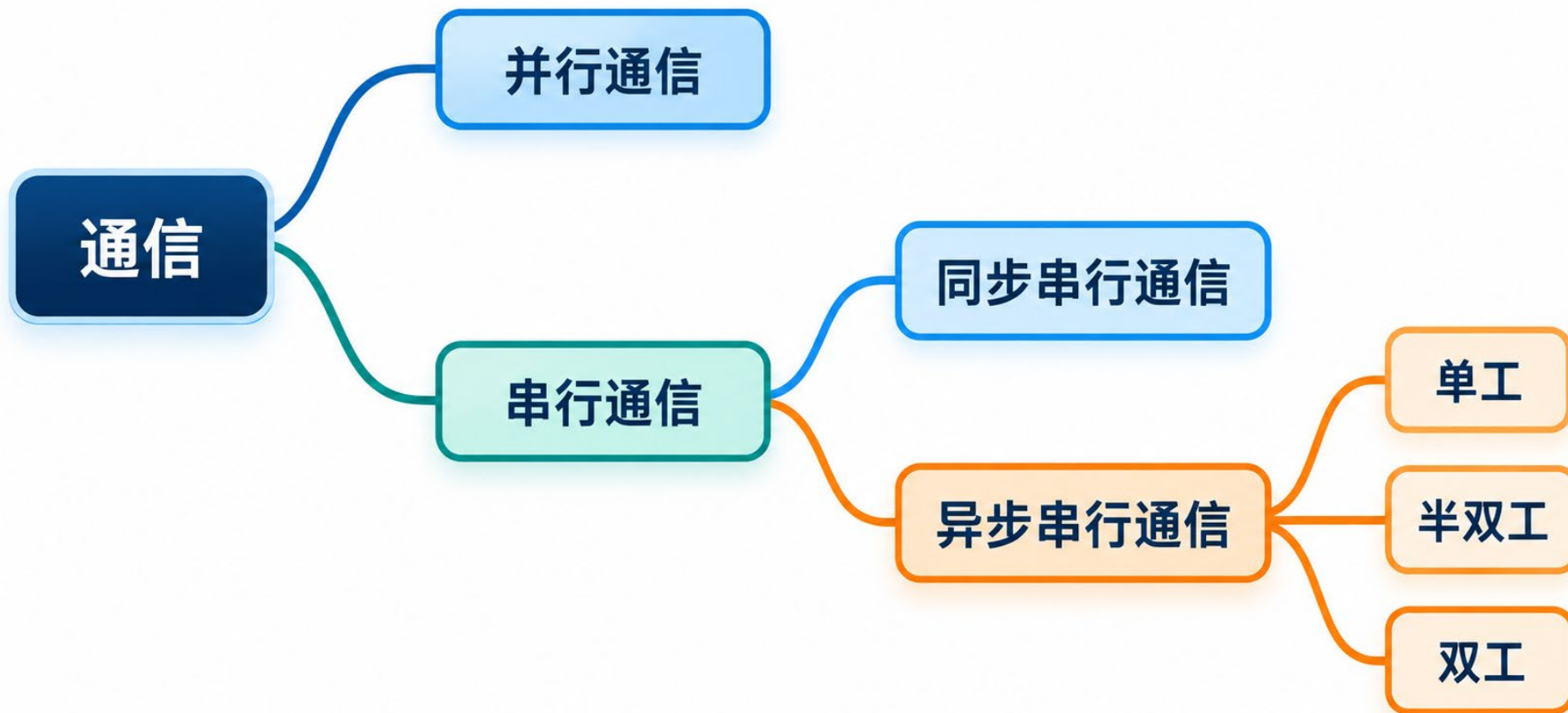
03 USART典型应用步骤



01 通信基本概念

嵌入式系统中的通信是指MCU与MCU或外围设备之间的信息交换。STM32系列微控制器提供了通用同步/异步收发器（USART, Universal Synchronous Asynchronous Receiver Transmitter）和低功耗通用异步收发器（LPUART, Low power universal synchronous asynchronous receiver transmitter）用于同步或异步通信。



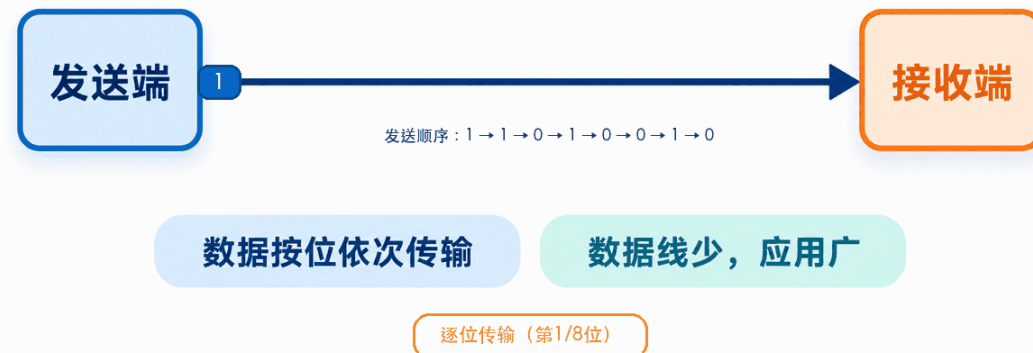


并行通信



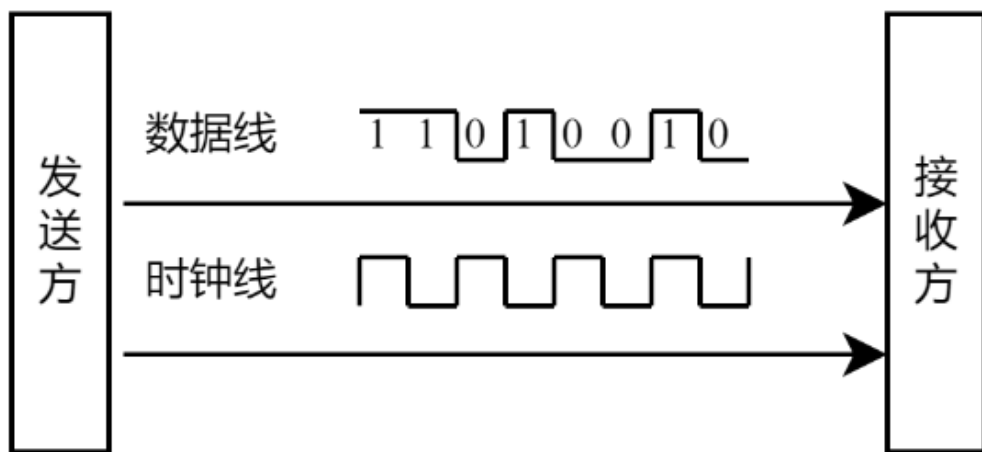
并行通信用多条数据线将数据字节中的各位**同时传输**。
效率高，成本高。

串行通信



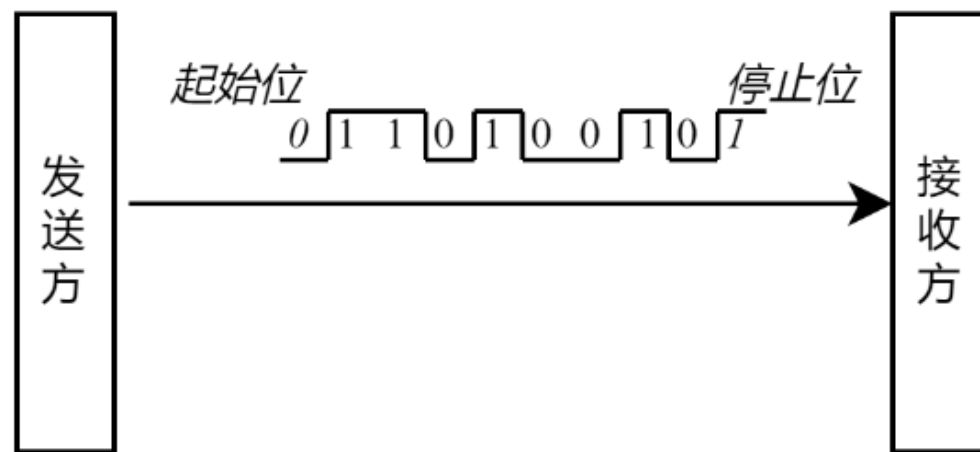
串行通信用一条数据线将数据字节中的各位**逐位传输**。
成本低，主流通信。

同步串行通信和异步串行通信



(a) 同步串行通信

有时钟线，传输信息帧（多个字符），用于芯片间通信。



(b) 异步串行通信

无时钟线，传输字符帧（一个字符），加起停标志，通信速率相同，用于设备通信。

按照数据传送方向，分为：

◆单工：

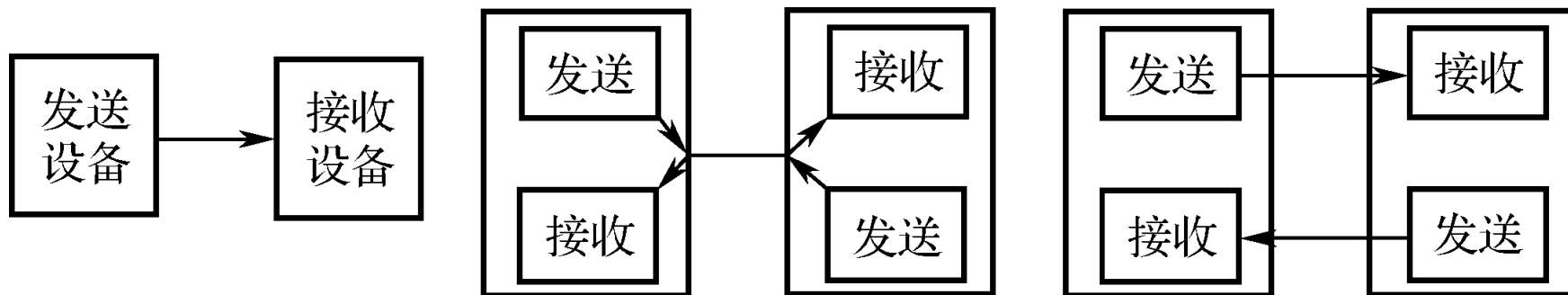
数据传输只支持数据在一个方向上传输；

◆半双工：

允许数据在两个方向上传输，但是，在某一时刻，只允许数据在一个方向上传输，它实际上是一种切换方向的单工通信；

◆全双工：

允许数据同时在两个方向上传输，因此，全双工通信是两个单工通信方式的结合，它要求发送设备和接收设备都有独立的接收和发送能力。



(a) 单工通信

(b) 半双工通信

(c) 全双工通信

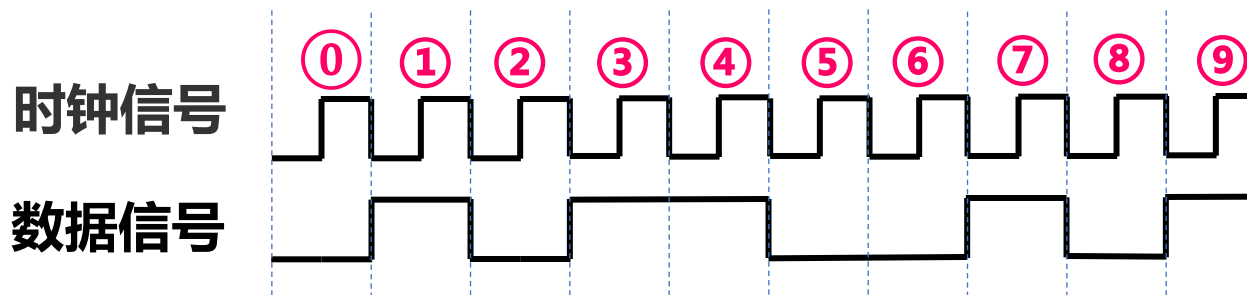
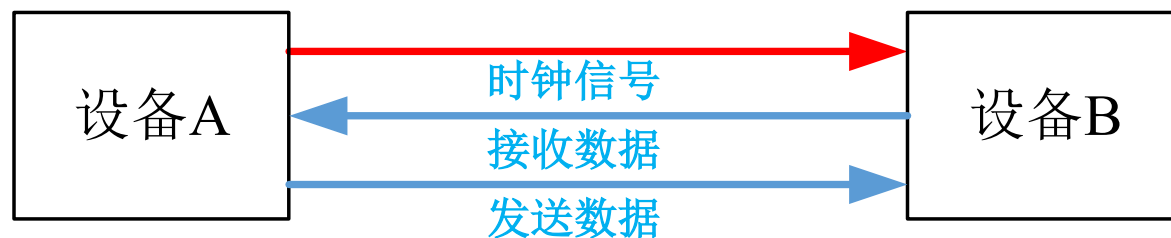
同步串行通信和异步串行通信

通信标准	引脚说明	通信方式	通信方向
UART (通用异步收发器)	TXD:发送端 RXD:接受端 GND:公共地	异步通信	全双工
USB	D+、D-	同步通信	半双工
SPI	SCK:同步时钟 MISO:主机输入，从机输出 MOSI:主机输出，从机输入	同步通信	全双工
I2C	SCL:同步时钟 SDA:数据输入/输出端	同步通信	半双工

同步串行通信和异步串行通信

1、同步通信：带时钟同步信号传输。

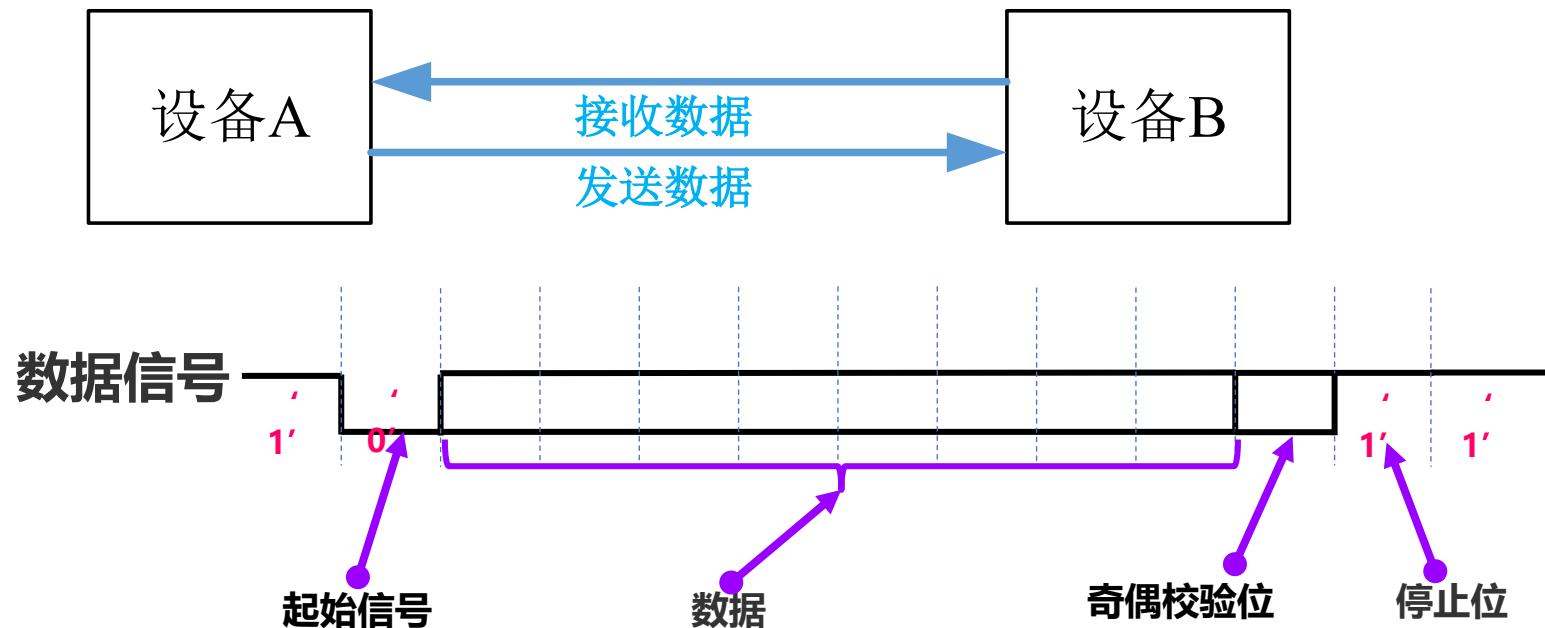
SPI, IIC通信接口, USB



同步串行通信和异步串行通信

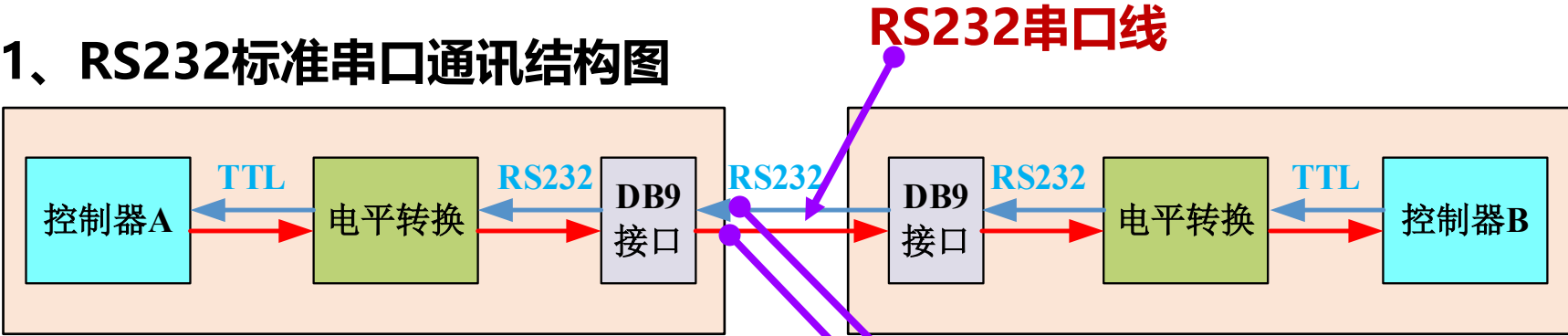
2、异步通信：不带时钟同步信号。

UART(通用异步收发器)



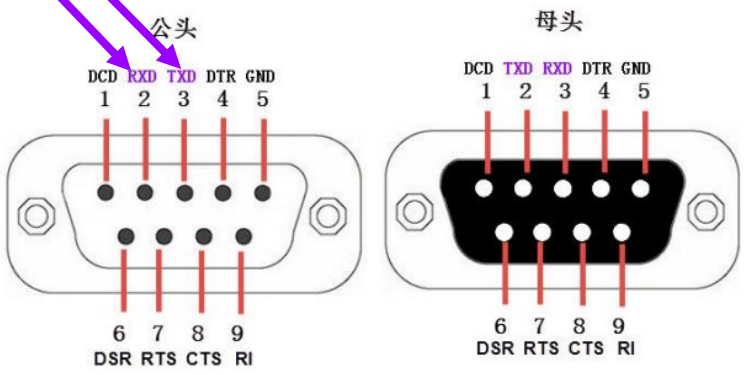
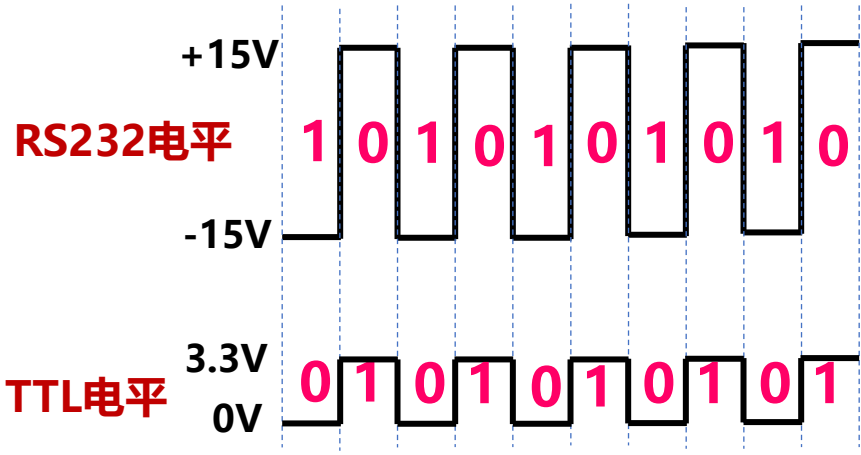
RS-232

1、RS232标准串口通讯结构图



1)、RS232标准串口主要用于工业设备直接通信

2)、电平转换芯片一般有MAX3232, SP3232



DB9 标准的公头 DB9 标准的母头

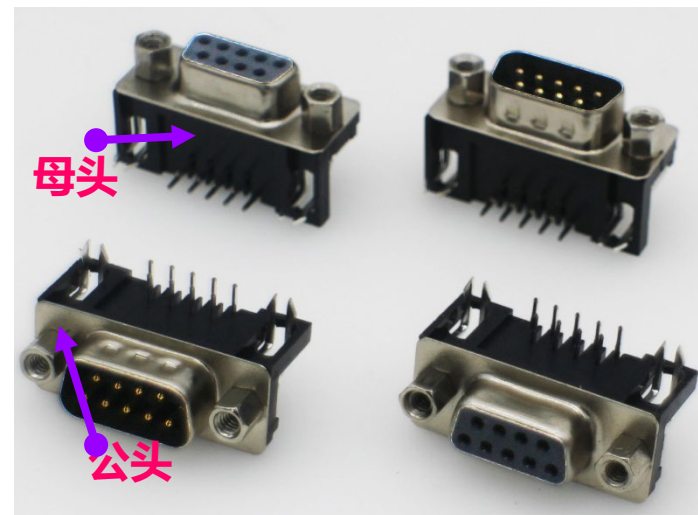
RS-232



DB9串口线



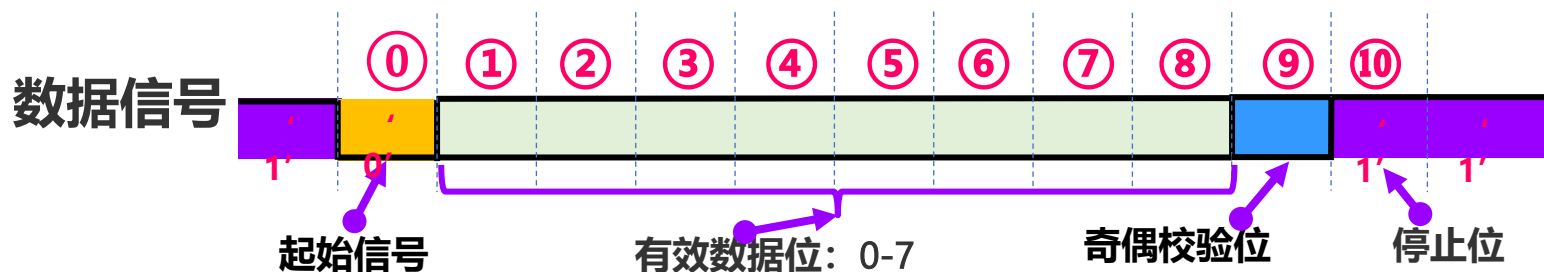
USB转串口线



- 1)、USB转串口主要用于设备跟电脑通信
- 2)、电平转换芯片一般有CH340、PL2303、CP2102、FT232
- 3)、使用的时候电脑端需要安装电平转换芯片的驱动

串口数据帧格式

串口数据包的基本组成



- 1) **起始位**: 由1个逻辑 0 的数据位表示;
 - 2) **结束位**: 由 0.5、 1、 1.5 或 2 个逻辑 1 的数据位表示;
 - 3) **有效数据**: 在起始位后紧接着的就是有效数据, 有效数据的长度常被约定为 5、 6、 7 或 8 位长;
 - 4) **奇偶校验位**: 可检验数据传输过程中是否出现错误。可选。
 - 5) **波特率(bps)**: 每秒传输的位数 (包括**起始位**、**停止位**、**有效数据位**、**奇偶校验位**) 。
- 常用波特率: 9600bps、 115200bps。

奇偶校验

校验方法分为：

1-奇校验(*odd*)、 2-偶校验(*even*)、 3-无校验(*noparity*)

1) 奇校验(*odd*)：有效数据和校验位中 ‘1’ 个数为奇数

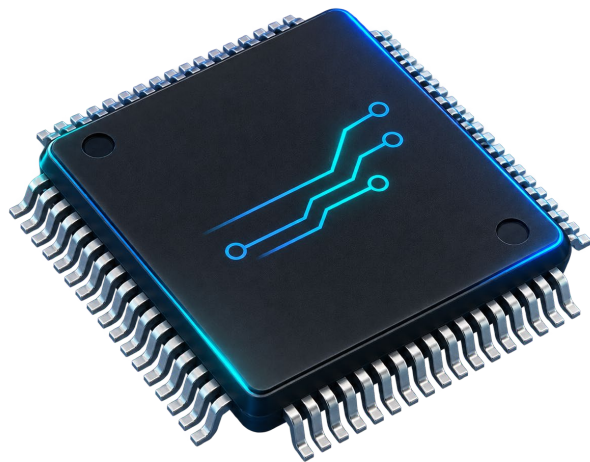
比如一个 8 位长的有效数据为：01101001，此时总共有 4 个 ‘1’，为达到奇校验效果，校验位为 ‘1’，最后传输的数据将是 8 位的有效数据加上 1 位的校验位总共 9 位

2) 偶校验(*even*)：有效数据和校验位中 ‘1’ 的个数为偶数

比如一个 8 位长的有效数据为：01101001，此时总共有 4 个 ‘1’，为达到偶校验效果，校验位为 ‘0’，最后传输的数据将是 8 位的有效数据加上 1 位的校验位总共 9 位。

02 STM32串口系统

STM32F407的串口资源



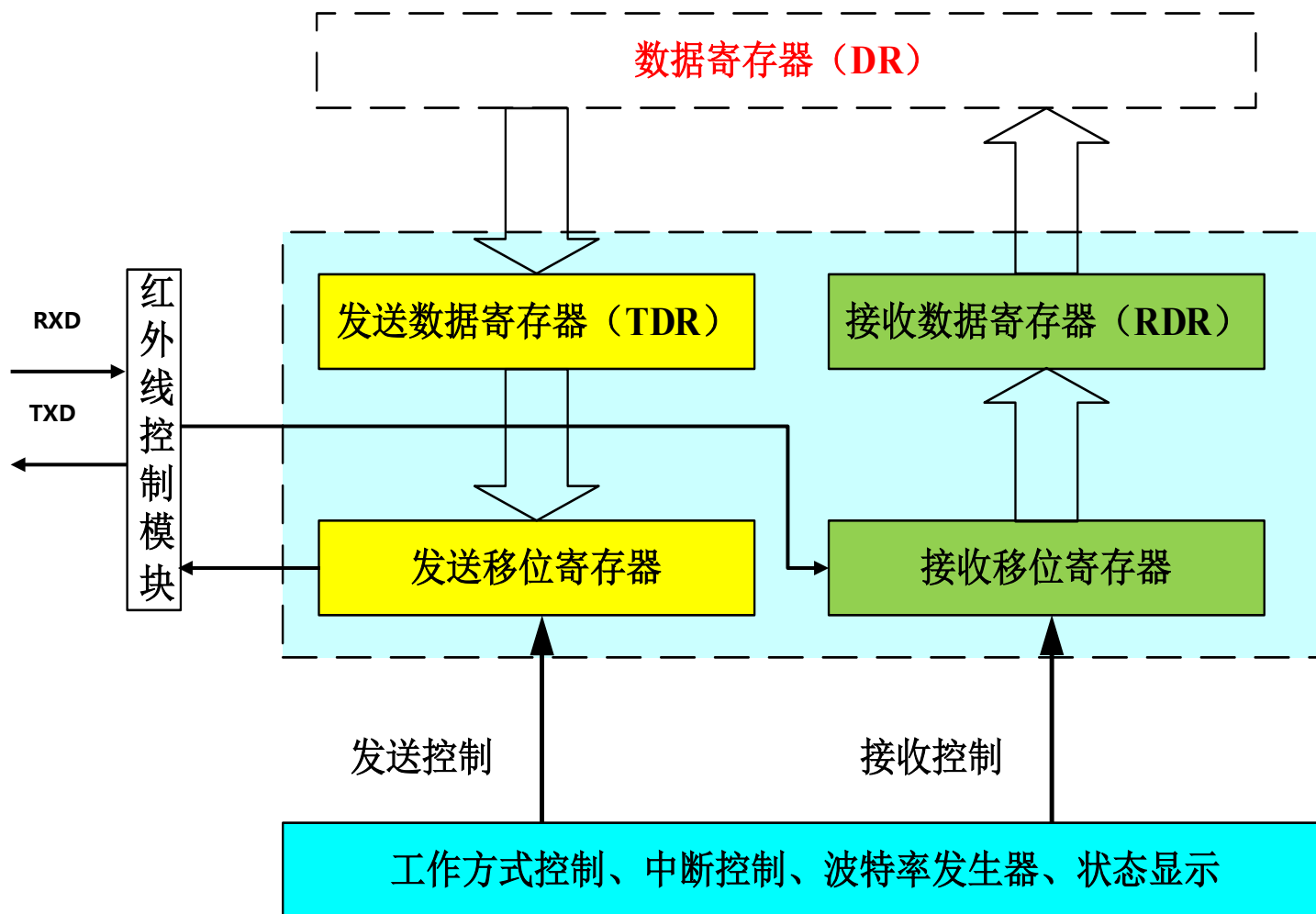
6

路串口资源

USART × 4 + UART × 2
USART 支持同步与异步，UART 主要用于异步通信

外设	总线	常用 TX / RX 引脚	复用
USART1	APB2	PA9/PA10 PB6/PB7	AF7
USART2	APB1	PA2/PA3 PD5/PD6	AF7
USART3	APB1	PB10/PB11 PC10/PC11 PD8/PD9	AF7
UART4	APB1	PA0/PA1 PC10/PC11	AF8
UART5	APB1	PC12/PD2	AF8
USART6	APB2	PC6/PC7 PG14/PG9	AF8

USART主要由**引脚**、**数据通道**、**发送器**、**接收器**等组成。



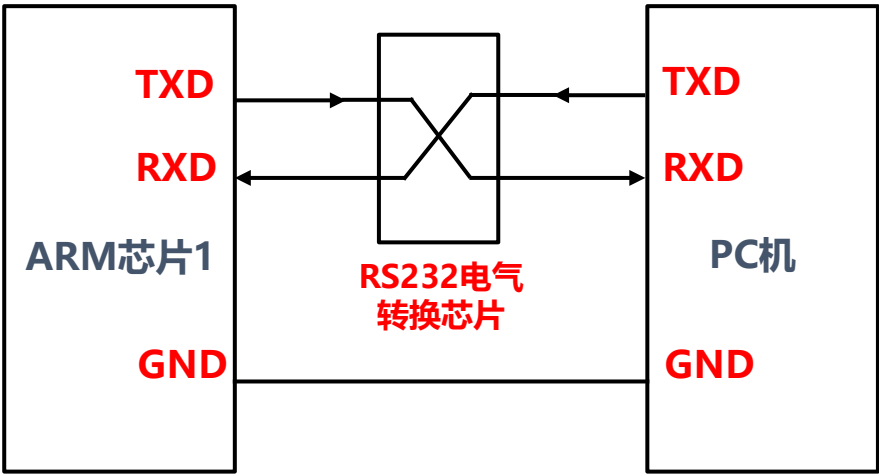
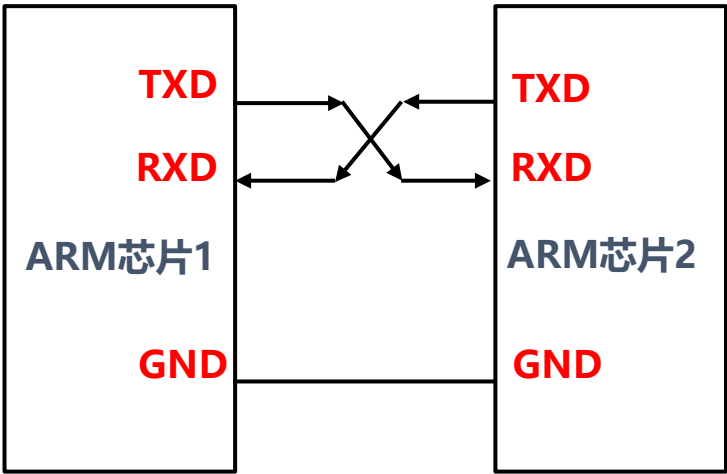
1 引脚

RXD: 数据接收引脚
TXD: 数据发送引脚
SCLK: 同步模式下时钟引脚

使用硬件流控时:
nCTS: nCTS表示清除已发送
nRTS: nRTS表示请求已发送

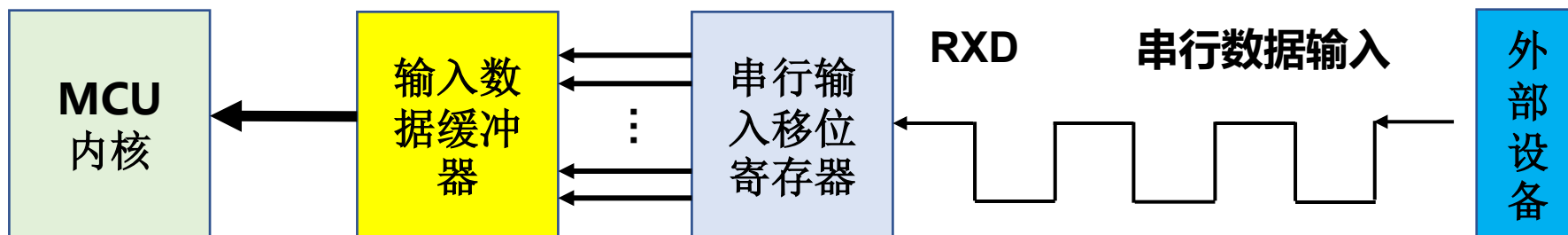
UART异步通信方式引脚(STM32F4):

串口号	RXD	TXD
1	PA10(PB7)	PA9 (PB6)
2	PA3(PD6)	PA2(PD5)
3	PB11(PC11/PD9)	PB10(PC10/PD8)
4	PC11(PA1)	PC10(PA0)
5	PD2	PC12
6	PC7(PG9)	PC6(PG14)

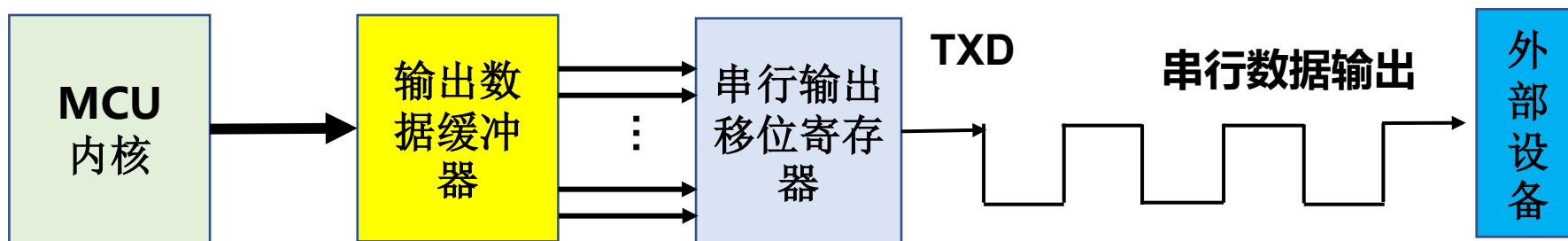


2 数据通道

数据接收过程:



数据发送过程:

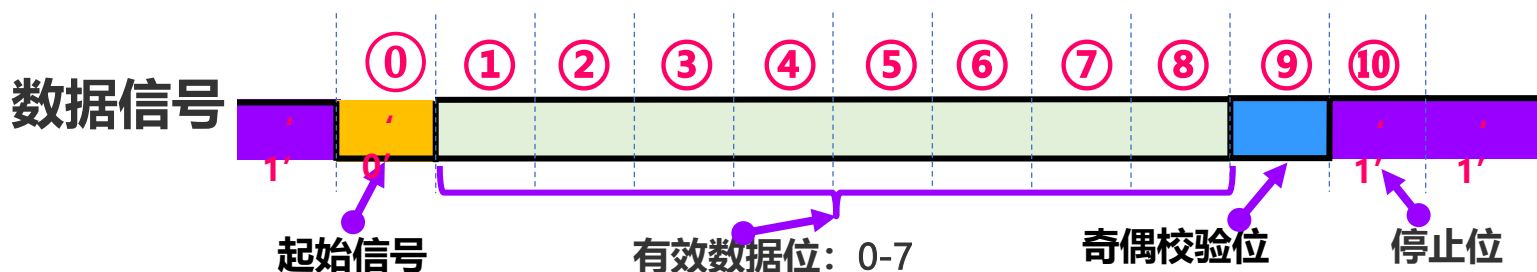


3 发送器

发送器控制USART的数据发送功能。发送数据由**起始位、有效数据位、校验位和停止位组成**。起始位一般为1位，有效数据位可以是7位或8位，校验位1位，停止位可以是0.5位、1位、1.5位和2位。

当USART_CR1发送使能位（TE）置1时，使能发送功能，**写USART_DR启动一次发送**，发送移位寄存器中的数据在TX引脚输出，首先发送LSB。

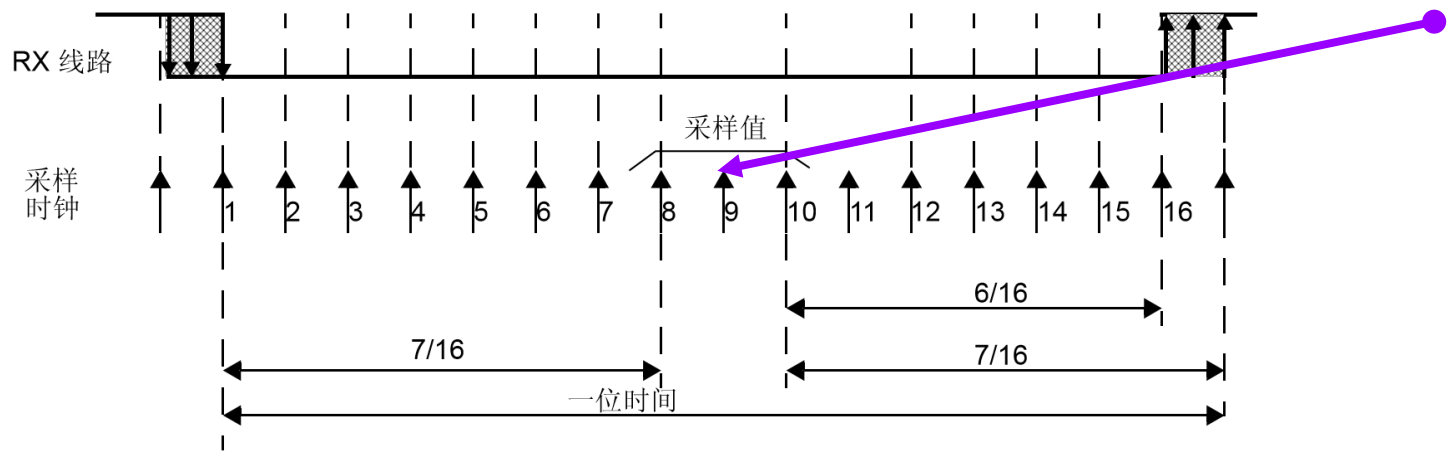
- ① 起始位
- ② 数据位（8位或者9位）
- ③ 奇偶校验位（第9位）
- ④ 停止位（1,1.5,2位）
- ⑤ 波特率设置



4 接收器

当USART_CR1的发送使能位 (RE) 置1时，使能接收功能。在USART接收期间，首先通过RX引脚移入数据的LSB。**当USART_SR的RXNE位置1，表明移位寄存器的内容已传送到RDR，数据接收结束。**

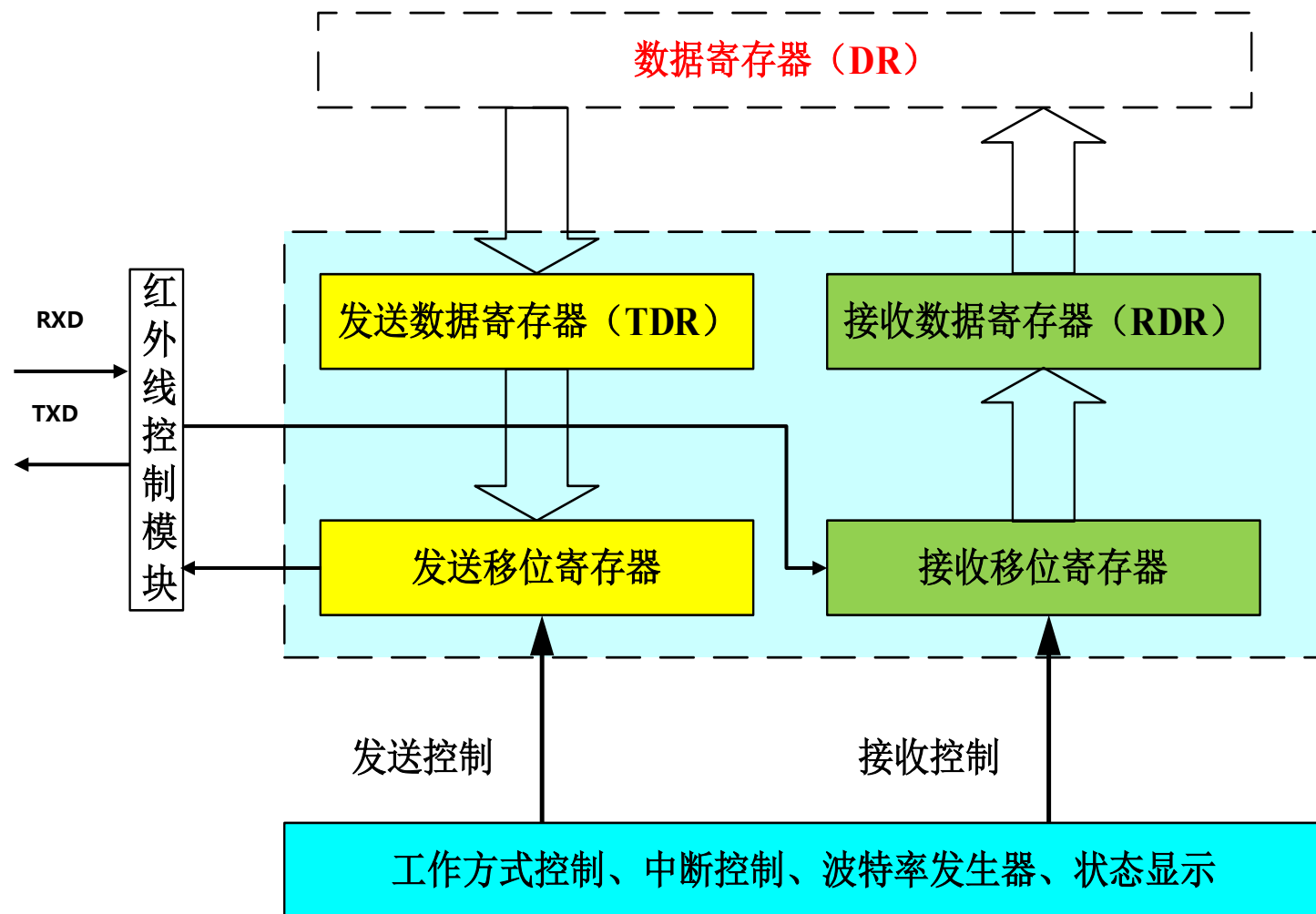
16倍过采样率数据位检测:



采样值	NE 状态	接收的位值
000	0	0
001	1	0
010	1	0
011	1	1
100	1	0
101	1	1
110	1	1
111	0	1

常用的串口相关寄存器

- 1)控制寄存器 (USART_CRx)
- 2)状态寄存器(USART_SR)
- 3)数据寄存器(USART_DR)
- 4)波特率寄存器(USART_BRR)



03 USART典型应用步骤

1 串口配置一般步骤

1、串口初始化

主要涉及到时钟配置、GPIO配置以及串口功能设置。

2、串口应用主要处理数据的接收和发送。

发送：常用软件查询法实现。

接收：常用中断法实现。

3、步骤：

① 串口时钟使能： `RCC_APBxPeriphClockCmd()` ;

GPIO时钟使能： `RCC_AHB1PeriphClockCmd()` ;

② 引脚复用映射： `GPIO_PinAFConfig()` ;

GPIO端口模式设置： `GPIO_Init()` ; 模式设置为 `GPIO_Mode_AF`

③ 串口参数初始化： `USART_Init()` ;

串口功能设置的核心函数。

1 串口配置一般步骤

④开启中断并且初始化NVIC (如果需要开启中断才需要这个步骤)

```
NVIC_Init();
```

```
USART_ITConfig();
```

⑤使能串口: `USART_Cmd();`

⑥编写中断处理函数: `USARTx_IRQHandler();`

串口数据收发:

```
void USART_SendData(); //发送数据到串口, DR
```

```
uint16_t USART_ReceiveData(); //接受数据, 从DR读取接受到的数据
```

串口传输状态获取:

```
FlagStatus USART_GetFlagStatus(); void USART_ClearFlag();
```

```
ITStatus USART_GetITStatus(); void USART_ClearITPendingBit();
```

2 常用库函数说明

stm32f4xx_usart.h

stm32f4xx_usart.c

1、串口功能初始化函数

```
void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* USART_InitStruct)
```

串口初始化：波特率，数据字长，奇偶校验，硬件流控以及收发使能

```
typedef struct
{
    __IO uint16_t SR; //状态寄存器
    uint16_t RESERVED0;
    __IO uint16_t DR; //数据寄存器
    uint16_t RESERVED1;
    __IO uint16_t BRR; //波特率寄存器
    uint16_t RESERVED2;
    __IO uint16_t CR1; //控制寄存器1
    uint16_t RESERVED3;
    __IO uint16_t CR2; //控制寄存器2
    uint16_t RESERVED4;
    __IO uint16_t CR3; //控制寄存器3
    uint16_t RESERVED5;
    __IO uint16_t GTPR; //保护时间和预分频器寄存器
    uint16_t RESERVED6;
} USART_TypeDef;
```

```
#define USART1 ((USART_TypeDef *) USART1_BASE)
#define USART6 ((USART_TypeDef *) USART6_BASE)

typedef struct
{
    uint32_t USART_BaudRate; //波特率
    uint16_t USART_WordLength; //有效数据位
    uint16_t USART_StopBits; //停止位数
    uint16_t USART_Parity; //校验方式
    uint16_t USART_Mode; //串口模式发送/接收/收发
    uint16_t USART_HardwareFlowControl; //硬件流控
} USART_InitTypeDef;
```

2 常用库函数说明

2、串口使能函数

```
void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState)
```

使能或禁止串口功能。

3、串口中断使能函数

```
void USART_ITConfig(USART_TypeDef* USARTx, uint16_t USART_IT,  
                    FunctionalState NewState)
```

使能或禁止串口中断功能。

```
#define USART_IT_TC      ((uint16_t)0x0626)  
#define USART_IT_RXNE   ((uint16_t)0x0525)
```

4、发送数据

```
void USART_SendData(USART_TypeDef* USARTx, uint16_t Data)
```

使能串口功能时，往DR寄存器写一个数据，启动一次发送过程。一般是一个字节，

5、接收数据

```
uint16_t USART_ReceiveData(USART_TypeDef* USARTx)
```

使能串口功能时，从DR寄存器读一个数据出来。一般是一个字节，

ENABLE或DISABLE



2 常用库函数说明

6、获取串口状态函数

```
#define USART_FLAG_TC      ((uint16_t)0x0040)  
#define USART_FLAG_RXNE   ((uint16_t)0x0020)
```

```
FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_FLAG)
```

```
ITStatus USART_GetITStatus(USART_TypeDef* USARTx, uint16_t USART_IT)
```

```
#define USART_IT_TC      ((uint16_t)0x0626)  
#define USART_IT_RXNE   ((uint16_t)0x0525)
```

7、清楚串口状态函数

```
void USART_ClearFlag(USART_TypeDef* USARTx, uint16_t USART_FLAG)
```

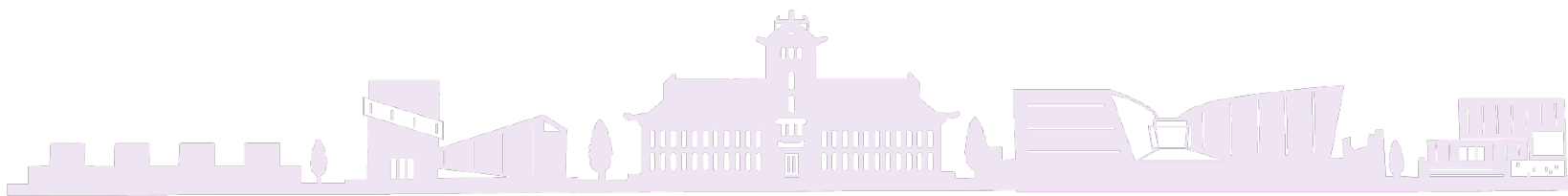
```
void USART_ClearITPendingBit(USART_TypeDef* USARTx, uint16_t USART_IT)
```

本章小结

01 通信基本概念

02 STM32串口系统

03 USART典型应用步骤



谢谢！